

Android Adware Detection Using Machine Learning and Deep Learning Techniques

Harini

Dept. of Computer Science and Engineering
IIITDM Jabalpur
Jabalpur, India

Mokshith

Dept. of Computer Science and Engineering
IIITDM Jabalpur
Jabalpur, India

Akhil

Dept. of Computer Science and Engineering
IIITDM Jabalpur
Jabalpur, India

Ranjith

Dept. of Computer Science and Engineering
IIITDM Jabalpur
Jabalpur, India

Chatur

Dept. of Computer Science and Engineering
IIITDM Jabalpur
Jabalpur, India

Abstract— Android malware has become one of the major cybersecurity threats due to the rapid growth of mobile applications and smartphone users. Among different malware categories, adware poses significant risks by displaying unwanted advertisements, collecting user information, and degrading device performance. To address this challenge, this paper proposes a hybrid machine learning and deep learning framework for Android adware detection using the CIC-MalDroid dataset, which comprises 244,630 samples (204,948 benign, 39,682 adware) with 85 behavioral features. The proposed system performs data preprocessing including median imputation, feature scaling, mutual information-based feature selection, and class balancing using SMOTE, followed by classification using Logistic Regression, Random Forest, XGBoost, and Artificial Neural Networks (ANN). Threshold optimization via Precision-Recall analysis is also employed. Experimental results show that Random Forest achieves the best overall performance with an accuracy of 87.12%, Macro-F1 of 0.7616, and ROC-AUC of 0.8781, followed by XGBoost with accuracy of 83.45%, Macro-F1 of 0.7139, and ROC-AUC of 0.8501. The proposed framework provides an efficient solution for Android adware detection and contributes toward improving mobile cybersecurity systems.

Index Terms— Android Malware, Adware Detection, Machine Learning, Deep Learning, Random Forest, XGBoost, Artificial Neural Network, CIC-MalDroid, SMOTE, Cybersecurity

1 INTRODUCTION

The rapid growth of Android smartphones and mobile applications has significantly increased the risk of cybersecurity threats in recent years. Android is one of the most widely used mobile

operating systems due to its open-source nature, flexibility, and large application ecosystem. Nonetheless, its widespread usage has made android smartphones one of the main victims of malware attacks. Malware has different types but adware has become quite notorious since it not only causes harm to the system security but also affects the privacy and performance of the users.

Adware is a kind of malicious application that causes the display of unsolicited advertisements, redirects users towards unreliable sites, and steals personal user information. Moreover, in most of the cases, it uses up the excess power of the device and slows down the battery efficiency as well. Traditionally used malware detection systems do not prove helpful in recognizing new malwares, which makes it necessary to use new methods of malware detection.

Recently, the use of machine learning and deep learning algorithms in detecting malware has increased tremendously because of their learning capabilities.

These techniques improve detection accuracy by automatically identifying malicious characteristics from application features. In this work, a hybrid machine learning and deep learning framework is proposed for Android adware detection using the CIC-MalDroid dataset. The dataset contains 244,630 samples with 85 behavioral network traffic features, distributed as 204,948 benign samples and 39,682 adware samples, exhibiting a natural class imbalance ratio of approximately 5.16:1.

The proposed system applies preprocessing techniques such as handling missing and infinite values via median imputation, feature scaling using StandardScaler, mutual information-based feature selection via SelectKBest, and SMOTE balancing.

prior to model training. After SMOTE is applied to the training split, both classes reach 163,958 samples each, enabling unbiased learning. The framework evaluates Logistic Regression, Random Forest, XGBoost, and Artificial Neural Network (ANN) models using probability threshold optimization alongside standard performance metrics including Accuracy, ROC-AUC Score, Macro-F1, F1-Benign, F1-Adware, Confusion Matrix, and Precision-Recall analysis.

The primary aim of this paper is to develop an optimal Android adware detection system that will help in enhancing malware classification and minimizing incorrect predictions. This proposed system has the potential to contribute towards the improvement of mobile cybersecurity and serve as a basis for future real-time malware detection systems.

This paper is structured as follows: the next section describes literature review. Proposed methodology is introduced in section 3. Experimental setup is provided in section 4. Implementation of models is described in section 5. Section 6 highlights the results and analysis. Conclusions and future work are discussed in the final section.

2 LITERATURE REVIEW

Detecting android malware has become an area of significance for research, owing to the growing threat of malicious applications aimed at android platforms. Various techniques for detecting malware have been proposed, both based on static analysis, dynamic analysis, and hybrid approaches. The traditional approach to malware detection was largely signature-based, where signatures of malware were matched with the application files. Although these techniques are effective for known malware, they fail to detect newly emerging and obfuscated malware variants.

To overcome these limitations, machine learning-based malware detection techniques have been introduced. Yerima et al. * demonstrated the use of Naive Bayes and Bayesian networks for Android malware classification, establishing early benchmarks for learning-based approaches. Machine learning algorithms capable of learning patterns from application behavior have subsequently improved upon these baselines. Several researchers have used classification algorithms such as Logistic Regression, Decision Trees, Random Forest, Support Vector Machines, and gradient boosting methods for Android malware detection, reporting consistent improvements over rule-based systems.

Deep learning techniques have also gained popularity in cybersecurity applications. Alzaylaee et al. * applied deep learning to dynamic analysis of Android applications and demonstrated that neural networks can identify behavioral signatures that evade static detection. Likewise, Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) have been used in malware detection applications for improving classification efficiency by capturing complex feature relationships from raw data in the form of network traffic or opcodes.

Several recent works have focused on evaluating the detection of Android

malwares using CIC-MalDroid benchmark. Lashkari et al. * discussed the methodology of development of the benchmark and provided behavioral features obtained through real traffic data. Several techniques like feature scaling, feature selection, and SMOTE oversampling were used by researchers in preprocessing of such datasets to solve issues of class imbalance.

Despite advancements in malware detection research field, there are certain challenges which persist. Many existing approaches exhibit high false positive rate, poor generalization on unseen malware data samples, and problems related to class imbalance in such datasets. Moreover, certain models need high computational power and are unable to provide real time detection. The current work attempts to overcome all the aforementioned challenges by developing multiple models and applying preprocessing, threshold optimization, and SMOTE techniques.

3 PROPOSED METHODOLOGY

The proposed Android adware detection framework is designed using a combination of machine learning and deep learning techniques. The overall workflow includes dataset collection, preprocessing, feature selection, data balancing, model training, threshold optimization, and performance evaluation. The proposed methodology aims to accurately classify Android applications as benign or malicious based on extracted behavioral network traffic features. Fig. 1 illustrates the high-level pipeline of the proposed framework.

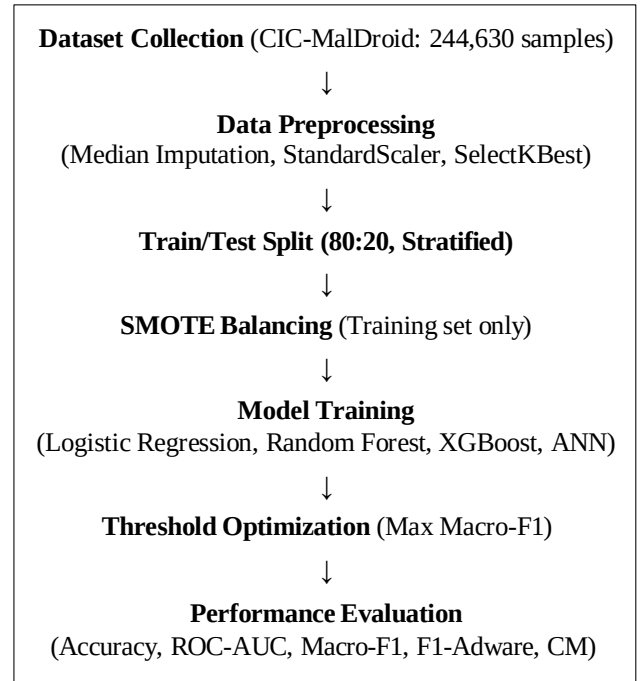


Figure 1: Proposed Android Adware Detection Framework Pipeline.

3.1 Dataset Collection

The CIC-MalDroid dataset was used for conducting the experiments in this work. The dataset contains Android network traffic and behavioral features collected from both benign applications and malicious adware samples. Separate CSV files containing adware and benign application data were loaded and processed programmatically. As shown in Fig. 2, the dataset scanning revealed 10 adware files and 311 benign files, totaling 321 CSV files. Common feature columns across all files were identified—85 columns in total—and both subsets were merged into a unified dataset.

```
Scanning files...
Adware files: 10
Benign files: 311
Finding common columns...
100%|██████████| 321/321 [00:14<00:00, 21.63it/s]

✓ Common columns found: 85

Loading files for label 1...
100%|██████████| 10/10 [00:02<00:00, 3.53it/s]

Loading files for label 0...
100%|██████████| 311/311 [00:06<00:00, 47.82it/s]

Merging datasets...
Dataset shape: (244630, 85)

Filtering numeric columns...
Shape after numeric filtering: (244630, 81)

Final Feature Shape: (244630, 80)
Final Label Shape: (244630,)

Class Distribution:
TARGET
0    204948
1     39682
Name: count, dtype: int64

scale_pos_weight = 5.16 (used in XGBoost)
```

Figure 2: Dataset loading and class distribution output. Total samples: 244,630 (Benign: 204,948; Adware: 39,682). scale_pos_weight for XGBoost = 5.16.

The merged dataset contains 244,630 samples and 85 features prior to numeric filtering. After retaining only numerical columns, 81 features remain, and after removing the target column, 80 features are used for model training. The class distribution reveals a significant imbalance: 204,948 benign samples (83.8%) versus 39,682 adware samples (16.2%), with an imbalance ratio of approximately 5.16:1.

3.2 Data Preprocessing

Data preprocessing was performed to improve data quality and model performance, as shown in Fig. 3. The dataset was split into training (195,704 samples) and testing (48,926 samples) sets using stratified 80:20 splitting before any preprocessing was applied to prevent data leakage. Infinite values were replaced with NaN and then handled using median imputation. Feature scaling was performed using StandardScaler. Feature selection was applied using SelectKBest with mutual information scoring, retaining the 80 most informative features for malware classification.

```
Splitting dataset...
Train shape: (195704, 80)
Test shape: (48926, 80)

Cleaning infinite values...

Handling missing values...

Scaling features...

Selecting important features via mutual information...
Shape after feature selection: (195704, 80)

Applying SMOTE (full balance: sampling_strategy=1.0)...
After SMOTE: Counter({0: 163958, 1: 163958})
```

Figure 3: Preprocessing pipeline output: train/test split, missing value handling, feature scaling, feature selection, and SMOTE balancing results.

The preprocessing pipeline is summarized in Table 1.

Table 1: Data Preprocessing Steps

Step	Technique / Result
Train/Test Split	80%/20% Stratified
Missing & Inf Value Handling	Median Imputation
Feature Scaling	StandardScaler
Feature Selection	SelectKBest (Mutual Information, $k=8$)
Data Balancing (Train only)	SMOTE (1:1 ratio $\rightarrow$ 163,958 each)

3.3 Dataset Balancing

Class imbalance is present in the data set (ratio 5.16:1 for benign vs adware). To counter this, SMOTE method is used only on the training set after splitting into train/test set. The SMOTE technique creates synthetic minority class samples through inter-polation between the current adware samples. Following SMOTE, there will be equal number of samples of both the classes in the training set (163,958 per each class – Counter: {0: 163958, 1: 163958}).

3.4 Machine Learning Models

Three classical algorithms for machine learning have been used: Linear Regression, which acts as a baseline classifier, Random Forest, which is a decision-tree based ensemble method, and XGBoost, which is a gradient boosting machine algorithm. The XGBoost algorithm was also tuned using parameters such as learning rate, scale_pos_weight=5.16, maximum depth, and subsampling ratio.

3.5 Deep Learning Model — ANN

Artificial Neural Network (ANN) model was built utilizing Tensor-Flow and Keras library. The ANN model contained several dense layers, and along with that, batch normalization and dropout layers were used. The sigmoid activation function was used in the output layer for binary classification purposes, and the threshold optimization was done for inference. Table 2 represents ANN architecture details..

Table 2: ANN Architecture Summary

Layer Type	Units	Details
Dense (Input)	256	ReLU activation
Batch Normalization	—	Normalize activations
Dropout	—	Rate = 0.3
Dense (Hidden)	128	ReLU activation
Batch Normalization	—	Normalize activations
Dropout	—	Rate = 0.3
Dense (Output)	1	Sigmoid activation

3.6 Threshold Optimization

Instead of employing the default cut-off value of 0.5, the optimal decision threshold for each of the models was calculated based on the maximization of Macro F1-Score through different threshold values on the Precision-Recall curve. The optimal values were found to be: Logistic Regression (0.511), Random Forest (0.505), XGBoost (0.831), and ANN (0.439). This particular process holds special significance in a security setting due to the cost of false alarms and missed alarms.

4 EXPERIMENTAL SETUP

The experiments were conducted using Google Colab as the primary development environment. Python 3.x was used for implementing all preprocessing, machine learning, and deep learning operations. The key libraries used included Pandas and NumPy for data manipulation, Scikit-learn for machine learning models, preprocessing, and SMOTE (via imbalanced-learn), TensorFlow/Keras for the ANN model, XGBoost for gradient boosting, and Matplotlib/Seaborn for visualization.

The CIC-MalDroid dataset was accessed through Google Drive integration within the Colab environment. The dataset was divided using an 80:20 stratified train/test split, yielding 195,704 training samples and 48,926 test samples. SMOTE was applied only to the training portion after the split. Table 3 summarizes the experimental configuration.

5 MODEL IMPLEMENTATION

5.1 Logistic Regression

Logistic Regression was implemented as a linear baseline model. Despite its simplicity, it provides a useful reference

Table 3: Experimental Configuration

Parameter	Value / Tool
Environment	Google Colab
Language	Python 3.x
Dataset	CIC-MalDroid
Total Samples	244,630 (Benign: 204,948; Adware: 39,682)
Features (after selection)	80
Train/Test Split	80% / 20% (Stratified)
After SMOTE (Train)	163,958 per class
Test Samples	48,926
Balancing	SMOTE (train only)
ML Libraries	Scikit-learn, XGBoost, imbalanced-learn
DL Libraries	TensorFlow 2.x, Keras

point for evaluating the benefit of more complex nonlinear models. With an optimal threshold of 0.511, its performance is limited on the highly imbalanced adware class, achieving F1-Adware of only 0.3362.

5.2 Random Forest

Random Forest was implemented as an ensemble learning algorithm. The model combines predictions from multiple decision trees to reduce variance and overfitting. Its ensemble nature makes it robust to noisy and high-dimensional network traffic data. With an optimal threshold of 0.505, Random Forest achieved the best overall performance in this study, with an accuracy of 87.12% and ROC-AUC of 0.8781. Feature importance scores from Random Forest also provide interpretability regarding the most discriminative behavioral features.

5.3 XGBoost

XGBoost was configured with hyperparameters including `scale_pos_weight=5.16` to explicitly account for class imbalance at the model level (complementing SMOTE at the data level). Additional parameters such as learning rate, maximum tree depth, subsampling ratio, and L1/L2 regularization were tuned for optimal performance. Despite its gradient boosting strength, the optimal threshold for XGBoost was found to be higher (0.831), indicating the model assigns higher confidence to adware predictions.

5.4 Artificial Neural Network

The ANN was trained for several epochs using batch normalization and dropout regularization to avoid overfitting. The optimal threshold for inference was set at 0.439 which is just below the default 0.5 because the model had been calibrated to detect positives early on. The ANN achieves a ROC-AUC of 0.6750 and Macro-F1 of 0.5646.

6 RESULTS AND DISCUSSION

The following subsections present detailed classification results for each model, derived directly from experimental outputs on the 48,926-sample test set.

6.1 Logistic Regression Results

Fig. 4 shows the classification report and confusion matrix for Logistic Regression. At optimal threshold 0.511, the model achieves an accuracy of 64.90%, Macro-F1 of 0.5488, and ROC-AUC of 0.6365. The model correctly classifies 27,407 benign samples but misclassifies 13,583, and correctly detects 4,348 adware samples while missing 3,588. The linear decision boundary is insufficient for capturing the complex nonlinear patterns in network traffic data.

```

=====
Model: Logistic Regression
=====
Optimal threshold: 0.511

Classification Report:
precision    recall  f1-score   support

Benign (0)    0.88    0.67    0.76    40990
Adware (1)    0.24    0.55    0.34     7936

   accuracy    0.56    0.61    0.65    48926
  macro avg    0.56    0.61    0.55    48926
weighted avg    0.78    0.65    0.69    48926

Confusion Matrix:
[[27407 13583]
 [ 3588  4348]]
ROC-AUC: 0.6365
Macro-F1: 0.5488

```

Figure 4: Logistic Regression classification report. Accuracy: 64.90%, ROC-AUC: 0.6365, Macro-F1: 0.5488. Confusion matrix: TN=27,407, FP=13,583, FN=3,588, TP=4,348.

6.2 Random Forest Results

Fig. 5 and Fig. 6 present the Random Forest classification report and confusion matrix. At threshold 0.505, Random Forest achieves accuracy of 87.12%, Macro-F1 of 0.7616, and ROC-AUC of 0.8781—the best performance of all evaluated models. The model correctly classifies 37,897 benign and 4,726 adware samples, with 3,093 false positives and 3,210 false negatives, demonstrating strong robustness on the imbalanced test set.

6.3 XGBoost Results

Fig. 7 shows the XGBoost output. At optimal threshold 0.831, XGBoost achieves an accuracy of 83.45%, Macro-F1 of 0.7139, and ROC-AUC of 0.8501. The high optimal threshold (0.831) reflects the model’s tendency toward high-confidence adware predictions, which is appropriate for conservative deployment scenarios. XGBoost is the second-best performing model, demonstrating competitive performance with excellent scalability.

6.4 ANN Results

Fig. 8 shows the ANN evaluation output. At optimal threshold 0.439, the ANN achieves an accuracy of 67.71%, Macro-F1 of

```

=====
Model: Random Forest
=====
Optimal threshold: 0.505

Classification Report:
precision    recall  f1-score   support

Benign (0)    0.92    0.92    0.92    40990
Adware (1)    0.60    0.60    0.60     7936

   accuracy    0.87    0.87    0.87    48926
  macro avg    0.76    0.76    0.76    48926
weighted avg    0.87    0.87    0.87    48926

Confusion Matrix:
[[37897  3093]
 [ 3210  4726]]
ROC-AUC: 0.8781
Macro-F1: 0.7616

```

Figure 5: Random Forest classification report. Accuracy: 87.12%, ROC-AUC: 0.8781, Macro-F1: 0.7616. Best performing model overall.

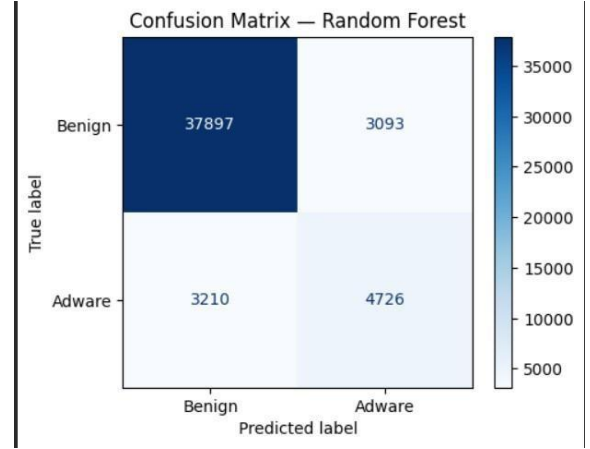


Figure 6: Confusion matrix for Random Forest. TN=37,897 (Benign correctly classified), FP=3,093, FN=3,210, TP=4,726 (Adware correctly detected).

```

=====
Model: XGBoost
=====
Optimal threshold: 0.831

Classification Report:
precision    recall  f1-score   support

Benign (0)    0.91    0.89    0.90    40990
Adware (1)    0.49    0.57    0.53     7936

   accuracy    0.70    0.73    0.83    48926
  macro avg    0.70    0.73    0.71    48926
weighted avg    0.85    0.83    0.84    48926

```

Figure 7: XGBoost classification report. Accuracy: 83.45%, ROC-AUC: 0.8501, Macro-F1: 0.7139. Optimal threshold: 0.831.

0.5646, and ROC-AUC of 0.6750. The ANN correctly classifies 28,999 benign and 4,129 adware samples, with 11,991 false positives and 3,807 false negatives. While the ANN’s overall accuracy is lower than the tree-based models, its learning capability demonstrates the potential for improvement with further architecture tuning.

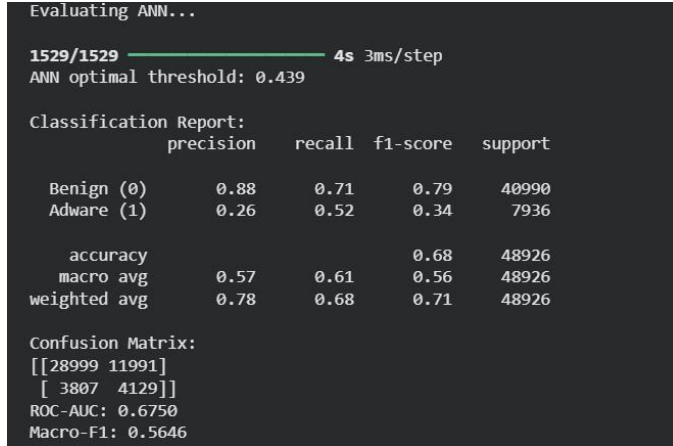


Figure 8: ANN classification report. Accuracy: 67.71%, ROC-AUC: 0.6750, Macro-F1: 0.5646. Optimal threshold: 0.439.

6.5 Comparative Model Performance

Table 4 presents the complete comparative performance summary across all models, directly derived from experimental results shown in Fig. 9.

Table 4: Comparative Model Performance (Test Set: 48,926 Samples)

Model	Thresh.	Acc.	Macro-F1	F1-Ben.	F1-Adw.	ROC-AUC
Log. Reg.	0.511	0.649	0.5488	0.7615	0.3362	0.6365
Rand. Forest	0.505	0.871	0.7616	0.9232	0.5999	0.8781
XGBoost	0.831	0.835	0.7139	0.8996	0.5282	0.8501
ANN	0.439	0.677	0.5646	0.7859	0.3433	0.6750

6.6 ROC Curve Analysis

Fig. 10 shows the ROC curves for all models. Random Forest (AUC=0.878) achieves the highest area under the ROC curve, followed by XGBoost (AUC=0.850), ANN (AUC=0.675), and Logistic Regression (AUC=0.636). The Random Forest curve rises steeply toward the upper-left corner, confirming its superior true positive rate at low false positive rates. XGBoost is competitive and particularly strong in the low-false-positive operating regime.

===== SUMMARY =====						
Model	Threshold	Accuracy	Macro-F1	F1-Benign	F1-Adware	ROC-AUC
Logistic Regression	0.511	0.6499	0.5488	0.7615	0.3362	0.6365
Random Forest	0.505	0.8712	0.7616	0.9232	0.5999	0.8781
XGBoost	0.831	0.8345	0.7139	0.8996	0.5282	0.8501
ANN	0.439	0.6771	0.5646	0.7859	0.3433	0.6750

Figure 9: Complete experimental summary table showing Threshold, Accuracy, Macro-F1, F1-Benign, F1-Adware, and ROC-AUC for all four models.

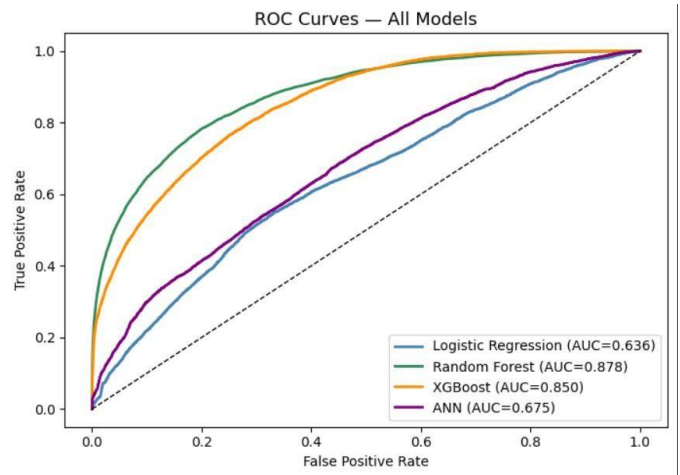


Figure 10: ROC curves for all models. Random Forest achieves AUC=0.878, XGBoost AUC=0.850, ANN AUC=0.675, and Logistic Regression AUC=0.636.

6.7 Precision-Recall Curve Analysis

Fig. 11 illustrates the Precision-Recall curves for all classifiers in regard to the adware (minority) class. In terms of precision, regardless of the level of recall, Random Forest outperforms other models. XGBoost exhibits decent precision in low levels of recall. On the other hand, both Logistic Regression and ANN exhibit poor Precision-Recall performance. This confirms the effectiveness of ensemble tree classifiers in the classification of the minority adware class.

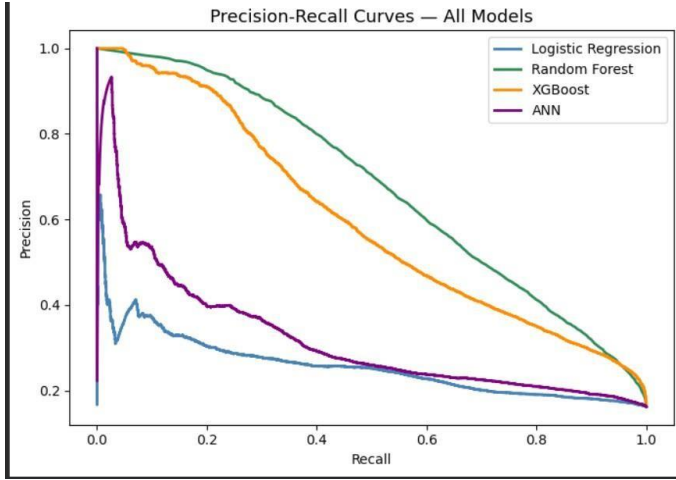


Figure 11: Precision-Recall curves for all models. Random Forest maintains the highest precision across recall values. XGBoost is competitive at lower recall levels.

6.8 Summary and Discussion

Some notable insights are gained from the experiments conducted. In this case, Random Forest has the highest performance due to its inherent ability to minimize variance as an ensemble technique through use of noisy network traffic data. XGBoost ranks second in terms of performance across all metrics. Logistic Regression and ANN were significantly out-performed by both methods.

The poor performance of ANN in comparison to the other techniques could be because of the few number of features (80) after the feature selection step and the heterogeneous nature of the traffic statistics. Threshold optimization proved beneficial for all models, especially XGBoost (threshold=0.831), enabling a favorable precision-recall trade-off in security-sensitive scenarios.

SMOTE balancing applied only to the training data was effective in preventing biased classification, while the stratified test set maintained the natural class distribution for realistic evaluation.

7 CONCLUSION

In this work, a hybrid machine learning and deep learning framework for Android adware detection was developed using the CIC-MalDroid dataset comprising 244,630 samples. Pre-processing techniques including median imputation, Standard-Scaler feature normalization, mutual information-based feature selection (80 features), and SMOTE balancing (applied exclusively to the training set) were employed to improve model performance and handle class imbalance.

Multiple models—Logistic Regression, Random Forest, XGBoost, and ANN—were implemented and evaluated using Accuracy, ROC-AUC Score, Macro-F1 Score, F1-Benign, F1-Adware, and Confusion Matrix analysis, all with probability threshold optimization. Among all models, Random Forest achieved the best classification performance with an accuracy of 87.12%, Macro-F1 of 0.7616, ROC-AUC of 0.8781, F1-Benign of 0.9232, and F1-Adware of 0.5999. XGBoost ranked second with accuracy of 83.45%, Macro-F1 of 0.7139, and ROC-AUC of 0.8501.

The outcomes of the experiments show that it is possible to successfully differentiate between benign and malicious Android apps using machine learning ensembles along with proper preprocessing and thresholds selection. Poor performance of Logistic Regression and ANN shows that nonlinear modeling is necessary in case of network traffic data.

Future Work

As for future work, efforts will be made to integrate the proposed approach into the context of dynamic analysis-based malware detection in Android. Indeed, dynamic analysis approaches could reveal the run-time behavior of malware, such as API calls, system calls, network communication behavior, as well as applications behavior, which is not possible using static analysis methods only. The introduction of behavioral analysis would likely increase the performance of detecting sophisticated adware types using obfuscation and evasion techniques.

Future research would include the creation of real-time dynamic malware analysis platforms, as well as the development of deep learning algorithms able to learn behavioral patterns during the malware execution process.

ACKNOWLEDGMENT

We sincerely acknowledge the Department of Computer Science and Engineering at IIITDM Jabalpur for providing us with the required facilities and infrastructure to perform this research work.

REFERENCES

1. Sharma, M.V.S., Tilak, M., Pareek,U. et al. Explainable AI-based Android Malware Detection.SN COMPUT. SCI. 6, 1006 (2025). <https://doi.org/10.1007/s42979-025-04548-3> Sharma, M.V.S., Tilak, M., Pareek,U. et al. Explainable AI-based Android Malware Detection. SN COMPUT. SCI. 6, 1006 (2025). <https://doi.org/10.1007/s42979-025-04548-3> link-<https://link.springer.com/article/10.1007/s42979-025-04548-3>
2. Mobile Malware Analysis ,Ms. K. Gowshika1 ,Akash A2, Ashwinprasad3, Kishor M V S4, Narendran A V5Mobile Malware Analysis Ms. K. Gowshika1, Akash A2, Ashwinprasad3, Kishor M V S4, Narendran A V5 link-https://www.ijaresm.com/uploaded_files/document_file/Ms_K_Gowshika_ZbS7.pdfhttps://www.ijaresm.com/uploaded_files/document_file/Ms_K_Gowshika_ZbS7.pdf
3. Android Malware Detectionusing Machine Learning,Ankashu Deepika1,S.Sreehaas2,Yerupula V. Pahvvan3, Ms. G. Mounica4Android Malware Detection using Machine Learning Ankashu De e pika1,S.Sre e haas,Ye rupulaV. Pahvvan3,Ms. G. Mounica link-https://www.ijaresm.com/uploaded_files/document_file/Ankashu_Deepika_FinalBt9p.pdfhttps://www.ijaresm.com/uploaded_files/document_file/Ankashu_Deepika_FinalBt9p.pdf
4. Priteshkumar Prajapati, Jugal Mehta, Prince Tadhani, Parth Shah, Amit Thakkar, Dipak Dabhi, "Detecting Malicious Android Apps Through Static FeaturesUsing Machine Learning", Information Systems for Intelligent Systems, vol.467, pp.311, 2026.Priteshkumar Prajapati, Jugal Mehta, Prince Tadhani, Parth Shah, Amit Thakkar, Dipak Dabhi, "DetectingMalicious Android Apps Through Static FeaturesUsing Machine Learning", Information Systems for IntelligentSystems, vol.467, pp.311, 2026. link-<https://ieeexplore.ieee.org/document/10459543/citations#><https://ieeexplore.ieee.org/document/10459543/citations#>
5. Abhishek Anand, Jyoti Prakash Singh, Vatsal Dhoundiyal, , 2024.Abhishek Anand, Jyoti Prakash Singh, Vatsal Dhoundiyal, , 2024. Esra KAVALCI YILMAZ, Halit BAKIR, "Hyperparameter Tunning and Feature SelectionMethods for Malware Detection",Journal of Polytechnic, 2023.Esra KAVALCI YILMAZ, Halit BAKIR, "Hype rparame te rTunning and Feature SelectionMethods for Malware Detection", Journal of Polytechnic, 2023. link-<https://ieeexplore.ieee.org/document/9914324/citations#><https://ieeexplore.ieee.org/document/9914324/citations#>